

Freight Logistics Architecture

Integrated Cloud Infrastructure & Secure Microservice Gateways

Technical Presentation

DevOps Group Systems Defense

1. System Overview & Core Requirements

- **Domain:** Mission-critical freight logistics web application.
- **Core Technical Priorities:**
 - Decoupled frontend static asset delivery from stateful compute layers.
 - Zero-downtime backend execution via warm App Service deployment slots.
 - Isolated microservice gateway via APIM for third-party integrations and testing.
- **Hosting Model:** Fully managed, cloud-native Azure ecosystem.

1b. Legacy Topology & Production Pain Points

- **High-Risk Promotions:** Had only a Test and Prod environment. Slot swaps were executed *between* Test and Prod directly, with no Staging layer for validation.
- **Monolithic Complications:** Combined UI and API deployment package. 3rd-party API calls were fired straight from the application runtime, stalling user interfaces when external systems lagged.
- **Severe Memory & Database Pressure:**
 - **No Caching Layer:** Typeahead relied on a massive, preloaded city-list object held entirely in API server memory.
 - **Storage Exhaustion:** Multi-city GPS coordinate paths sat inside a bloated database table. No partitioning on `AuditLogs` resulted in

2. Decoupled Multi-Environment Topology

```
graph TD
    subgraph Frontend_Layer [Direct CDN Publishing]
        SWA_Test[Angular Test SWA]
        SWA_Stag[Angular Staging SWA]
        SWA_Prod[Angular Prod SWA]
    end

    subgraph Backend_App_Service_Plan [Sequential Swap Chain]
        SWA_Test -->|Direct UI-to-API| API_Test[[Test Slot]]
        SWA_Stag -->|Direct UI-to-API| Slot_Stag[[Staging Slot]]
        SWA_Prod -->|Direct UI-to-API| Slot_Prod[[Production Slot]]

        API_Test <-->|1. PROMOTION SWAP| Slot_Stag
        Slot_Stag <-->|2. DEPLOYMENT SWAP| Slot_Prod
    end

    subgraph Relational_Data_Layer
        API_Test --> DB_Test[(Azure SQL: Test)]
        Slot_Stag -->|Slot-Sticky Setting| DB_Prod_Main[(Azure SQL: Prod)]
        Slot_Prod -->|Slot-Sticky Setting| DB_Prod_Main
    end
```

3. End-to-End Enterprise Identity & Security

- **Frontend Authentication:** The Angular UI integrates **MSAL (Microsoft Authentication Library)** to handle secure user authentication directly via Microsoft login.
- **The Token Lifecycle:**
 - MSAL acquires and automatically caches cryptographic JWT Access Tokens in the browser.
 - An internal HTTP interceptor attaches the token to all direct backend API requests.
- **Backend Authorization:** The `.NET Web API` validates the incoming Bearer token claims against **Azure Entra ID** to enforce granular Role-Based Access Control (RBAC).

4. Component Communication & Integration Architecture

```
graph TD
    Client[Angular Frontend / Azure SWA] -->|Direct HTTPS API Calls with MSAL Token| AppService[Web API: Azure App Services / Slots]

    subgraph Integrated_Storage_and_Caching [Integrated Storage & Caching]
        AppService -->|Cache Aside| Redis[(Azure Cache for Redis)]
        AppService -->|Relational State| SQL[(Azure SQL Database)]
        AppService -->|Docs & GPS Blobs| Blob[(Azure Blob Storage)]
    end

    subgraph Secure_Gateway_Facade [Secure Gateway Facade]
        AppService -->|Outbound Proxy / MTLS| APIM[Azure API Management]
        PM[Postman Client / Testing] -->|Direct Route Integration| APIM
        APIM <-->|OAuth2 / JWT Auth| Entra[Azure Entra ID]
        APIM -->|Internal Route| AzFunc[Azure Functions]
    end

    AzFunc -->|Webhooks / Polling| External[3rd Party Freight/Carrier APIs]
```

5. Compute State & Schema Continuity

- **Frontend Delivery:** Angular static web assets are directly published to their respective Azure SWA instances. No slot switches are performed on the CDN edge.
- **Backend Zero-Downtime Swaps:** API code is published to the **Staging Slot** and fully warmed up by hitting the root runtime path prior to traffic routing redirection.
- **Database Drift Defense:** Employing an **Expand and Contract pattern**. Schema migrations are pushed *before* the slot swap occurs using backward-compatible, non-breaking mutations.

6. Database Tuning & Cloud Cost Controls

To prevent unbounded data growth and optimize compute costs within Azure SQL, the relational layer implements two lifecycle architectures:

- **Horizontal Table Partitioning:**

- High-volume `AuditLogs` are partitioned by **date ranges**.
- Query engines leverage **partition pruning** to completely bypass irrelevant data blocks.
- Result: Sub-second user access to historic compliance trails.

- **Time-Bounded Log Retention:**

- System telemetry and API `UsageLogs` are capped on a rolling retention window.

7. Comprehensive 4-Tier Caching Topology

To aggressively maximize performance, caching layers are partitioned by data change frequency:

Tier	Caching Engine	Targeted Data Payload	Lifespan / Isolation Strategy
Tier 1	.NET ResponseCache	Long-term Static Lookups	Completely local in-memory; zero network hops.
Tier 2	Azure Cache for Redis	High-frequency Typeahead Search	Prefixed key spaces (<code>prod:typeahead:*</code>).
Tier 3	Memcached	High-frequency Static Content	Completely local in-memory; zero network hops.
Tier 4	CDN	Static Content	Completely local in-memory; zero network hops.

8. Tiered Route Resolution Pipeline

When a user requests multi-city route GPS coordinates, the Web API processes the request through a strict **3-tier failover lifecycle**:

```
graph TD
    Req[Incoming Route Request] --> T1{.NET ResponseCache}
    T1 -->|Hit: Valid < 24 Hours| Return[Return JSON Payload]

    T1 -->|Miss / Expired| T2{Azure Blob Storage}
    T2 -->|Hit: File Found| Warm[.NET Re-caches + Return]

    T2 -->|Miss: File Not Found| T3[Azure Maps API Compute]
    T3 --> Save[Write JSON to Blob Storage]
    Save --> Warm
```

9. Q&A and Engineering Appendix

- Open for peer review regarding schema management, token validation lifetimes, and caching boundaries.
- **Deep dives available in repository:**
 - Appendix A: System Component Trade-off Matrix
 - Appendix B: Dynamic Slot Swap & State Boundary Rules
 - Appendix C: Cache Key & Storage Lifecycle Strategy

